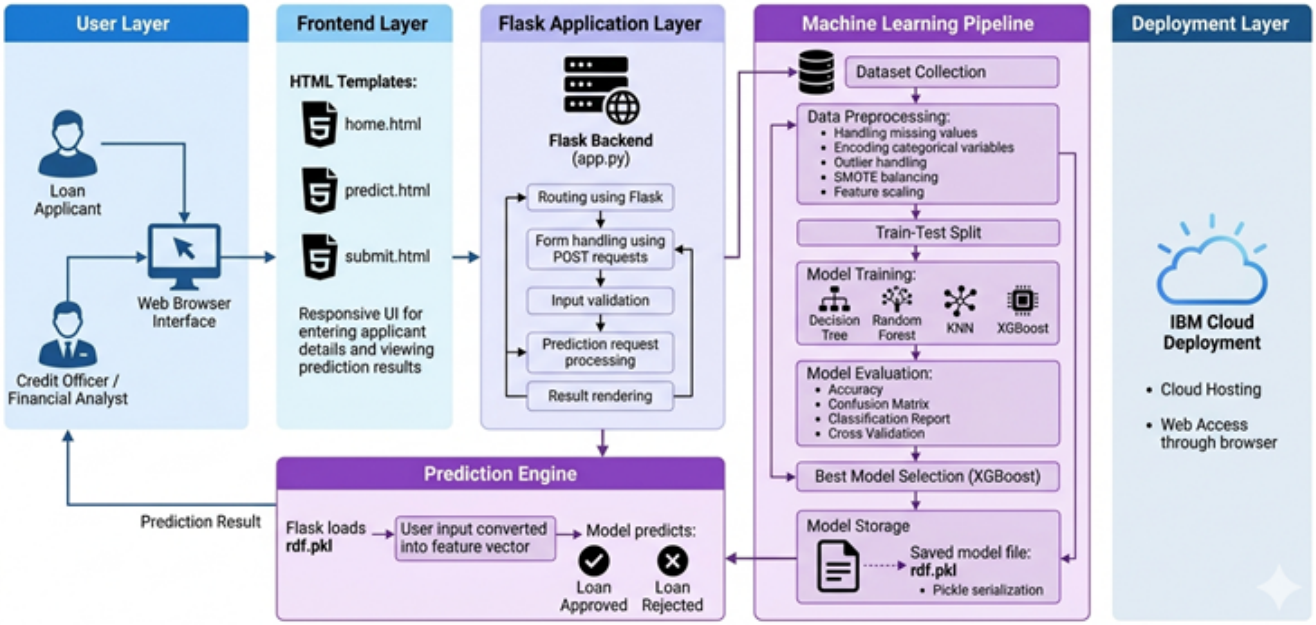
SMART LENDER

Applicant Credibility Prediction for Loan Approval

Machine Learning Powered Creditworthiness Prediction Web Application



Smart Lender Technical Architecture

Technology Stack: Python, Flask, HTML, CSS, Machine Learning, XGBoost, Scikit-learn, Pandas, NumPy, Pickle, IBM Cloud

Domain: Banking and Financial Services / Credit Risk Analytics

Project Description

Smart Lender is a machine learning-powered web application designed to predict the creditworthiness of loan applicants and support banks and financial institutions in making faster, consistent, and data-driven loan approval decisions. The application analyzes structured applicant information and predicts whether a loan is likely to be approved or rejected.

The system evaluates important applicant attributes such as gender, marital status, education, self-employment status, applicant income, co-applicant income, loan amount, loan amount term, credit history, and property area. These factors are transformed into machine-readable features and passed to trained classification models.

During the model development phase, four classification algorithms—Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost—are trained and evaluated. Based on comparative performance, XGBoost is selected as the best-performing model, achieving **94.7% training accuracy and 81.1% testing accuracy**. The selected model is serialized as a **.pkl** file and integrated with a Flask application for real-time prediction.

Built using Python and Flask and designed for deployment on IBM Cloud, Smart Lender provides a simple browser-based interface. A loan applicant, credit officer, or financial analyst can enter applicant details and instantly receive a prediction result. The solution can reduce repetitive manual evaluation, assist in early risk identification, and improve the speed of the credit assessment process.

Scenarios

Scenario 1: Fast-Track Approval for Low-Risk Applicants

A bank credit officer enters the details of a salaried applicant with stable income and a good credit history. The model predicts loan approval, helping the officer prioritize the application for faster processing and reduce unnecessary manual review.

Scenario 2: High-Risk Applicant Detection

An applicant with irregular self-employment income and weak or unavailable credit history submits a loan request. The model predicts a higher risk of rejection, prompting the credit team to perform additional document verification and detailed scrutiny before taking a final decision.

Scenario 3: High-Volume Applicant Evaluation

A financial analyst evaluates many applications during a high-volume processing period. By using the XGBoost prediction engine as a decision-support tool, the analyst can reduce initial screening time while maintaining a consistent evaluation approach.

Technical Architecture

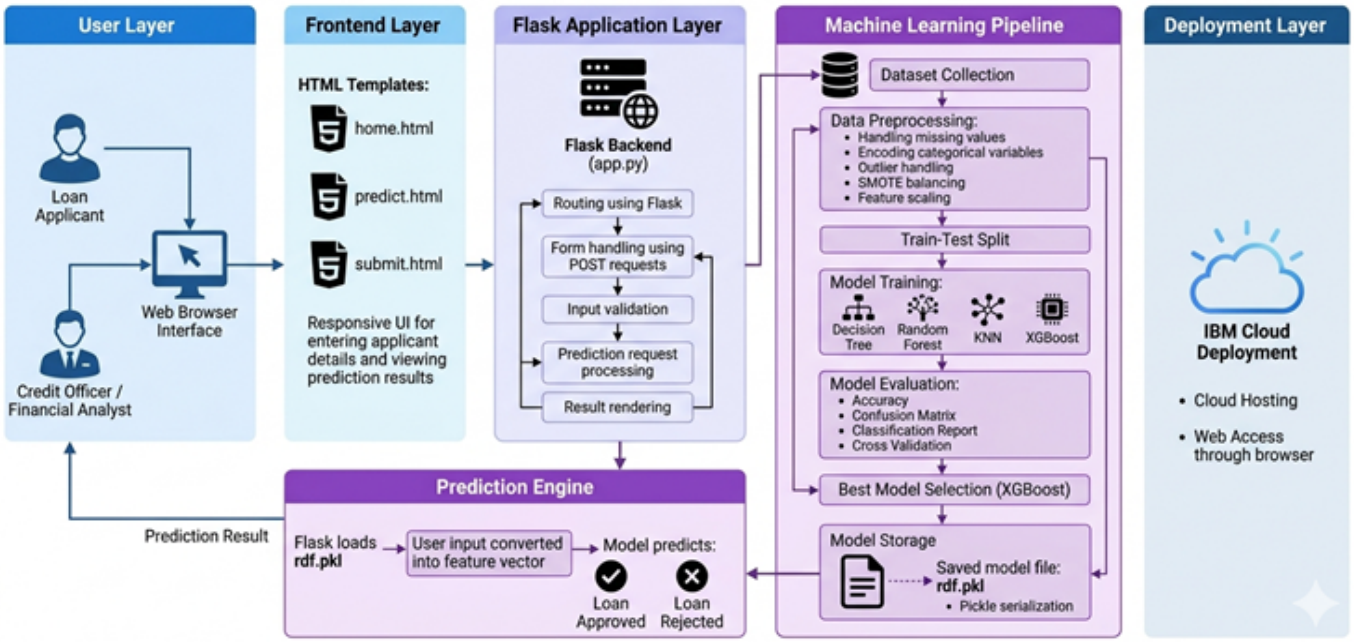


Figure 1: Smart Lender layered architecture and machine learning workflow

Description: Smart Lender follows a layered architecture consisting of the User Layer, Frontend Layer, Flask Application Layer, Machine Learning Pipeline, Prediction Engine, and Deployment Layer. The browser-based frontend collects applicant details through HTML templates. Flask manages routing, POST requests, input validation, prediction requests, and result rendering.

The machine learning pipeline performs dataset collection, missing-value handling, categorical encoding, outlier handling, SMOTE balancing, feature scaling, train-test splitting, model training, and evaluation. Decision Tree, Random Forest, KNN, and XGBoost are compared using accuracy, confusion matrix, classification report, and cross-validation. The best model is stored using Pickle. During prediction, Flask loads the saved model, converts user input into the expected feature vector, generates an approval or rejection prediction, and displays the result in the browser.

Pre-requisites

- Python programming knowledge and understanding of functions, libraries, and file handling.
- Basic machine learning knowledge: classification, training/testing data, model evaluation, and overfitting.
- Flask framework knowledge for routing, form handling, templates, and application execution.
- HTML and CSS skills for developing the loan applicant input interface and result pages.
- Pandas and NumPy knowledge for structured data processing and numerical operations.
- Scikit-learn knowledge for preprocessing, model training, metrics, and data splitting.
- XGBoost familiarity for gradient-boosted classification.
- Git and GitHub knowledge for version control and project management.
- IBM Cloud familiarity for cloud deployment and browser-based access.

Project Workflow

Milestone 1: Dataset Collection and Architecture

- Understand the loan approval problem and identify input and target variables.
- Collect and inspect the structured loan applicant dataset.
- Define the layered architecture connecting frontend, Flask, ML pipeline, prediction engine, and IBM Cloud deployment.
- Set up the Python development environment and install required libraries.

Milestone 2: Data Preprocessing and Exploratory Analysis

- Handle missing values and inconsistent records.
- Encode categorical variables and process numerical attributes.
- Detect and handle outliers where appropriate.
- Balance the target classes using SMOTE and prepare scaled features.

Milestone 3: Model Training and Evaluation

- Split the processed dataset into training and testing sets.
- Train Decision Tree, Random Forest, KNN, and XGBoost classifiers.
- Evaluate all models using accuracy, confusion matrix, classification report, and cross-validation.
- Select XGBoost as the best model based on comparative results.

Milestone 4: Flask Application Development

- Create Flask routes and HTML templates.
- Develop the applicant details form and validate submitted input.
- Load the saved model and connect the prediction engine to Flask.
- Render Loan Approved or Loan Rejected results.

Milestone 5: Deployment

- Prepare dependencies and saved model files for deployment.
- Configure the Flask application for cloud hosting.
- Deploy the Smart Lender application on IBM Cloud.
- Verify browser access and real-time prediction workflow.

Milestone 6: Conclusion

- Summarize project outcomes, limitations, and future enhancement opportunities.

Milestone 1: Dataset Collection and Architecture

The first milestone focuses on understanding the loan approval prediction problem, studying the dataset, and defining the overall system architecture. The objective is to establish a clear connection between the business problem and the machine learning solution.

Activity 1.1: Understand the Project Requirements

- Identify the target task as a binary classification problem: loan approval or loan rejection.
- Identify the intended users: loan applicants, credit officers, financial analysts, and banking professionals.
- Define the applicant information required by the model, including demographic, income, loan, credit, and property attributes.
- Define the expected output as an immediate prediction that supports, rather than replaces, institutional review and policy checks.

Activity 1.2: Collect and Inspect the Dataset

The dataset contains structured records of loan applicants. Each row represents an applicant and each column represents a characteristic used for analysis or prediction.

Feature	Purpose
Gender / Married	Applicant demographic information
Education	Graduate or non-graduate category
Self_Employed	Employment type indicator
ApplicantIncome	Primary applicant income
CoapplicantIncome	Co-applicant income
LoanAmount	Requested loan amount
Loan_Amount_Term	Repayment term
Credit_History	Historical credit repayment indicator
Property_Area	Urban, semi-urban, or rural area
Loan_Status	Target approval outcome

Activity 1.3: Define the Application Architecture

- **User Layer:** Loan applicants, credit officers, and financial analysts access the system through a web browser.
- **Frontend Layer:** HTML templates such as home.html, predict.html, and submit.html collect inputs and display results.
- **Flask Application Layer:** app.py handles routes, POST requests, input validation, prediction requests, and result rendering.
- **Machine Learning Pipeline:** Dataset preprocessing, train-test split, model training, model evaluation, and best-model selection are performed.
- **Prediction Engine:** Flask loads the saved model, converts submitted values into a feature vector, and predicts approval or rejection.
- **Deployment Layer:** IBM Cloud hosts the application and makes it accessible through a browser.

Activity 1.4: Set Up the Development Environment

```
python -m venv myenv
myenv\Scripts\activate
pip install flask pandas numpy scikit-learn xgboost imbalanced-learn
pip install matplotlib
pip freeze > requirements.txt
```

A suggested project structure is shown below:

```
Smart-Lender/
|-- app.py
|-- model/
| |-- rdf.pkl
|-- templates/
| |-- home.html
| |-- predict.html
| |-- submit.html
|-- static/
| |-- css/
| |-- style.css
|-- notebook/
| |-- model_training.ipynb
|-- requirements.txt
|-- Procfile / deployment files
```

Milestone 2: Data Preprocessing and Exploratory Analysis

Raw applicant data cannot always be used directly for model training. Missing values, categorical labels, outliers, and class imbalance can affect the learning process. This milestone transforms the raw dataset into a clean and consistent feature set.

Activity 2.1: Handle Missing Values

- Inspect every column using null-value counts.
- For numerical variables, use an appropriate statistical replacement such as median or mode depending on the feature distribution.
- For categorical variables, replace missing values with the most frequent category where appropriate.
- Verify that required model features contain no unresolved null values before training.

```
df.isnull().sum()

# Example approach
df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].median())
df['Credit_History'] = df['Credit_History'].fillna(df['Credit_History'].mode()[0])
```

Activity 2.2: Encode Categorical Variables

Machine learning models require numerical input. Categorical values such as Gender, Married, Education, Self_Employed, Property_Area, and Loan_Status are converted into numerical representations.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
for col in categorical_columns:
df[col] = le.fit_transform(df[col])
```

Activity 2.3: Handle Outliers

Numerical features such as ApplicantIncome, CoapplicantIncome, and LoanAmount are inspected for extreme values. Depending on the observed distribution, outliers may be capped, transformed, or retained when they represent valid financial cases.

Activity 2.4: Class Balancing and Feature Scaling

SMOTE Balancing: If the approval and rejection classes are imbalanced, the model may favor the majority class. Synthetic Minority Over-sampling Technique (SMOTE) creates synthetic examples for the minority class in the training data to improve learning balance.

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)
```

Feature Scaling: Numerical variables can have different ranges. Standardization is especially useful for distance-based algorithms such as KNN.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_resampled)
```

Activity 2.5: Feature Selection

The final model input is prepared using relevant applicant attributes. Unnecessary identifiers such as Loan_ID should not be treated as predictive financial features. The selected features must appear in the same order during both model training and Flask prediction.

Activity 2.6: Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
X_scaled, y_resampled,
test_size=0.20,
random_state=42,
stratify=y_resampled
)
```

The training set is used to learn model patterns, while the testing set is used to estimate performance on unseen records.

Milestone 3: Model Training and Evaluation

In this milestone, multiple classification algorithms are trained on the processed dataset. Comparing several models helps identify the algorithm that provides the best balance of predictive performance and generalization.

Activity 3.1: Decision Tree

Decision Tree creates a tree-like set of decision rules by splitting applicant records based on feature values. It is easy to interpret but can overfit the training data when the tree becomes too complex.

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)
```

Activity 3.2: Random Forest

Random Forest combines predictions from multiple decision trees. By averaging many trees, it generally reduces overfitting and improves robustness compared with a single Decision Tree.

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
```

Activity 3.3: K-Nearest Neighbors (KNN)

KNN classifies a new applicant by examining the classes of nearby training records in the feature space. Because it relies on distance, feature scaling is important.

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
```

Activity 3.4: XGBoost Model Training

XGBoost is a gradient boosting algorithm that builds decision trees sequentially. Each new tree focuses on correcting errors made by previous trees. It is effective for structured tabular datasets and can model complex relationships between applicant attributes.

```
from xgboost import XGBClassifier

xgb = XGBClassifier(
random_state=42,
eval_metric='logloss'
)
xgb.fit(X_train, y_train)

train_accuracy = xgb.score(X_train, y_train)
test_accuracy = xgb.score(X_test, y_test)

print('Training Accuracy:', train_accuracy)
print('Testing Accuracy:', test_accuracy)
```

For the Smart Lender project, the selected XGBoost model achieved **94.7% training accuracy** and **81.1% testing accuracy** based on the reported project results.

Activity 3.5: Model Evaluation

- **Accuracy:** Measures the percentage of correct predictions.
- **Confusion Matrix:** Shows correct and incorrect predictions for approval and rejection classes.
- **Classification Report:** Provides precision, recall, and F1-score.
- **Cross-Validation:** Evaluates model stability across multiple data splits.

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.model_selection import cross_val_score

print(accuracy_score(y_test, xgb.predict(X_test)))
print(confusion_matrix(y_test, xgb.predict(X_test)))
print(classification_report(y_test, xgb.predict(X_test)))

cv_scores = cross_val_score(xgb, X_scaled, y_resampled, cv=5)
print(cv_scores.mean())
```

Activity 3.6: Best Model Selection and Model Storage

After training Decision Tree, Random Forest, KNN, and XGBoost, their evaluation results are compared. XGBoost is selected as the best-performing model for the Smart Lender prediction engine based on the reported project performance.

Model	Role in Comparison
Decision Tree	Interpretable baseline classifier
Random Forest	Ensemble model for improved robustness
KNN	Distance-based classifier
XGBoost	Selected best model for deployment

The trained model is serialized using Pickle so that the Flask application can load it without retraining the algorithm every time the server starts.

```
import pickle

with open('rdf.pkl', 'wb') as file:
pickle.dump(xgb, file)

# Loading the model
with open('rdf.pkl', 'rb') as file:
model = pickle.load(file)
```

Important: The preprocessing logic used during training must also be applied to live user input. Feature encoding, scaling, and column order must remain consistent between training and prediction.

Milestone 4: Flask Application Development

Milestone 4 integrates the trained machine learning model with a Flask web application. Flask acts as the communication layer between the browser interface and the prediction engine.

Activity 4.1: Define Flask Routes

```
from flask import Flask, render_template, request
import pickle
import numpy as np

app = Flask(__name__)

with open('rdf.pkl', 'rb') as file:
model = pickle.load(file)

@app.route('/')
def home():
return render_template('home.html')

@app.route('/predict')
def predict_page():
return render_template('predict.html')

@app.route('/submit', methods=['POST'])
def submit():
# Read, validate and transform form values
# Create feature vector
# Run model prediction
# Render prediction result
pass

if __name__ == '__main__':
app.run(debug=True)
```

Activity 4.2: Process Form Input

The prediction form collects applicant values and submits them to Flask using a POST request. Each field is retrieved from request.form, validated, converted to the required numerical representation, and arranged in the exact feature order expected by the model.

```
gender = request.form.get('gender')
married = request.form.get('married')
education = request.form.get('education')
self_employed = request.form.get('self_employed')
applicant_income = float(request.form.get('applicant_income'))
coapplicant_income = float(request.form.get('coapplicant_income'))
loan_amount = float(request.form.get('loan_amount'))
loan_term = float(request.form.get('loan_term'))
credit_history = float(request.form.get('credit_history'))
property_area = request.form.get('property_area')
```

Activity 4.3: Prediction Engine

After validation and encoding, the submitted values are converted into a feature vector. The saved model receives the vector and returns a predicted class.

```
features = np.array([[
gender_encoded,
married_encoded,
education_encoded,
self_employed_encoded,
applicant_income,
coapplicant_income,
loan_amount,
loan_term,
credit_history,
property_area_encoded
]])

prediction = model.predict(features)[0]

if prediction == 1:
result = 'Loan Approved'
else:
result = 'Loan Rejected'

return render_template('submit.html', prediction_text=result)
```

If a scaler or other preprocessing object was fitted during training, the same saved transformer must be applied before model.predict().

Activity 4.4: Input Validation

- Check that all required fields are submitted.
- Reject empty or invalid numerical values.
- Restrict categorical fields to supported options.
- Validate that income, loan amount, and loan term values are within meaningful ranges.
- Display a user-friendly error message instead of exposing server errors.

Prediction Flow

User enters details → Browser sends POST request → Flask validates input → Values are encoded and converted to feature vector → Saved model predicts → Flask renders Loan Approved or Loan Rejected.

Milestone 5: Frontend Development

The frontend provides a simple interface for entering applicant information and viewing prediction results. HTML templates are rendered by Flask and styled using CSS.

Activity 5.1: Home Page

The home page introduces Smart Lender and explains that the application uses machine learning to support applicant credibility prediction. A call-to-action guides the user to the prediction form.

Activity 5.2: Prediction Form

- Gender
- Marital status
- Education
- Self-employment status
- Applicant income
- Co-applicant income
- Loan amount
- Loan amount term
- Credit history
- Property area

Dropdowns should be used for categorical fields and number inputs should be used for numerical fields. Clear labels and placeholders make the form easier to understand.

Activity 5.3: Result Page

After the form is submitted, the result page displays the model prediction. A positive prediction is shown as **Loan Approved**, while a negative prediction is shown as **Loan Rejected**. The result page should also provide a button to return to the prediction form.

Sample HTML Form Structure

```
&lt;form action="/submit" method="POST"&gt;
&lt;select name="gender" required&gt;...&lt;/select&gt;
&lt;select name="married" required&gt;...&lt;/select&gt;
&lt;input type="number" name="applicant_income" required&gt;
&lt;input type="number" name="loan_amount" required&gt;
&lt;select name="credit_history" required&gt;...&lt;/select&gt;
&lt;button type="submit"&gt;Predict Loan Status&lt;/button&gt;
&lt;/form&gt;
```

Milestone 6: Deployment on IBM Cloud

The deployment phase makes the Flask application accessible through a web browser. The Smart Lender architecture is designed for IBM Cloud hosting.

Activity 6.1: Prepare the Application

- Verify that app.py runs correctly in the local environment.
- Store the trained model file in the project and confirm the model path.
- Generate requirements.txt with all Python dependencies.
- Disable development-only settings before production deployment.
- Use environment variables for secrets and cloud-specific configuration.
- Confirm that the application listens on the host and port expected by the cloud runtime.

```
pip freeze > requirements.txt
python app.py
```

Activity 6.2: Cloud-Ready Flask Configuration

```
import os

if __name__ == '__main__':
port = int(os.environ.get('PORT', 5000))
app.run(host='0.0.0.0', port=port)
```

Activity 6.3: Deploy and Verify

- Create or configure the required IBM Cloud application service.
- Upload or push the Smart Lender project files.
- Install dependencies from requirements.txt during build.
- Start the Flask application using the configured runtime command.
- Open the deployed application URL in a browser.
- Submit test applicant records and verify approval/rejection predictions.
- Review application logs if a route, model file, or dependency fails.

Testing and Verification

Testing ensures that the web interface, Flask routes, preprocessing logic, model integration, and prediction result work together correctly.

Test Case	Input / Condition	Expected Result
TC-01	All valid applicant details	Prediction is generated
TC-02	Required field is empty	Validation message is displayed
TC-03	Invalid numeric value	Input is rejected safely
TC-04	Good credit history and stable inputs	Model returns its predicted class
TC-05	Riskier applicant pattern	Model returns its predicted class
TC-06	Prediction page opened	Form renders correctly
TC-07	POST request to submit route	Flask processes the request
TC-08	Saved model unavailable	Controlled server/log error handling
TC-09	Cloud URL opened	Application is accessible in browser

Model Verification

The four candidate models are compared using standard classification metrics. The selected XGBoost model reports 94.7% training accuracy and 81.1% testing accuracy in the project results. The difference between training and testing performance should be considered during model review and future tuning.

Important Responsible-Use Note

Smart Lender should be treated as a decision-support application. A production lending system must also follow applicable banking policies, legal requirements, fairness reviews, explainability requirements, data protection controls, and human oversight. Sensitive or protected attributes should be reviewed carefully before real-world use.

Exploring the Website's Web Pages

Home Page

Description: The home page introduces Smart Lender as an applicant credibility prediction system for loan approval. It explains the purpose of machine learning-based evaluation and provides navigation to the prediction page.

[Insert Smart Lender Home Page screenshot here]

Prediction Page

Description: The prediction page contains the applicant details form. Users select categorical values and enter numerical financial information. The form submits data to the Flask backend for validation and prediction.

[Insert Prediction Form screenshot here]

Prediction Result Page

Description: The result page displays the prediction generated by the trained XGBoost model. The user sees either Loan Approved or Loan Rejected and can return to the form to evaluate another applicant.

[Insert Loan Approved / Loan Rejected screenshot here]

Conclusion

Smart Lender is a machine learning-powered loan applicant credibility prediction system developed using Python, Flask, and classification algorithms. The project demonstrates the complete machine learning application lifecycle: dataset understanding, preprocessing, categorical encoding, outlier handling, class balancing with SMOTE, feature scaling, train-test splitting, model training, evaluation, best-model selection, model serialization, Flask integration, frontend development, and cloud deployment.

Decision Tree, Random Forest, K-Nearest Neighbors, and XGBoost are evaluated during model development. Based on the reported project results, XGBoost is selected as the best-performing model with 94.7% training accuracy and 81.1% testing accuracy. The trained model is saved as a Pickle file and integrated with Flask to provide real-time loan approval predictions through a browser-based interface.

The project can assist credit officers and financial analysts in performing faster initial applicant screening and identifying potentially risky cases for further review. By automating repetitive prediction steps, Smart Lender demonstrates how machine learning can support data-driven processes in banking and financial services.

Future Enhancements

- Add explainable AI techniques such as SHAP to show the factors influencing a prediction.
- Add secure user authentication and role-based access for credit officers and analysts.
- Store prediction history in a secure database with audit logging.
- Introduce batch applicant evaluation using CSV upload.
- Monitor model drift and retrain the model using reviewed, current data.
- Perform fairness testing and remove inappropriate dependence on sensitive attributes.
- Add stronger validation, encryption, and privacy controls for financial data.
- Develop dashboards for approval trends, risk segments, and model performance.
- Create a production-grade CI/CD and IBM Cloud deployment pipeline.

Final Outcome: Smart Lender provides a practical demonstration of integrating machine learning with a Flask web application to deliver real-time loan approval predictions and support faster, structured credit evaluation.

Project Summary

Project Name	Smart Lender – Applicant Credibility Prediction for Loan Approval
Domain	Banking and Financial Services / Credit Risk Analytics
Problem Type	Binary Classification
Algorithms	Decision Tree, Random Forest, KNN, XGBoost
Selected Model	XGBoost
Reported Training Accuracy	94.7%
Reported Testing Accuracy	81.1%
Backend	Python, Flask
Frontend	HTML, CSS
ML Libraries	Pandas, NumPy, Scikit-learn, XGBoost, imbalanced-learn
Model Storage	Pickle (.pkl)
Deployment Target	IBM Cloud
Primary Users	Loan Applicants, Credit Officers, Financial Analysts

This documentation was structured to follow the milestone-and-activity style of the supplied sample project documentation while adapting the content to the Smart Lender architecture and project details.